Implementation of Blocks World Program

```
from collections import deque


# State = tuple of stacks (bottom -> top); each stack sits on the table.
def normalise(stacks):
    return tuple(sorted(tuple(s) for s in stacks if s))


def blocks_moves(state):
    stacks = [list(s) for s in state]
    for i, src in enumerate(stacks):
        block = src[-1]
        # move onto another stack
        for j, dst in enumerate(stacks):
            if i != j:
                new = [list(s) for s in stacks]
                new[i].pop()
                new[j].append(block)
                yield f"Move {block} from {src[-2] if len(src) > 1 else 'table'} onto
                    {dst[-1]}", normalise(new)
        # move to table
        if len(src) > 1:
            new = [list(s) for s in stacks]
            new[i].pop()
            new.append([block])
            yield f"Move {block} from {src[-2]} to table", normalise(new)


def blocks_world(start, goal):
    start, goal = normalise(start), normalise(goal)
    queue, seen = deque([(start, [])]), {start}
    while queue:
        state, plan = queue.popleft()
        if state == goal:
            return plan
        for action, nxt in blocks_moves(state):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, plan + [(action, nxt)]))
    return None


def show_state(state):
    height = max(len(s) for s in state)
    for level in range(height - 1, -1, -1):
        print("   " + "  ".join(s[level] if level < len(s) else " " for s in state))
    print("   " + "--" * len(state) * 2)


def program10():
    print("\n=== 10. BLOCKS WORLD ===")
    start = [["A", "C"], ["B"]]           # C is on A, B alone on table
    goal = [["C", "B", "A"]]              # A on B on C
    print("Start:"); show_state(normalise(start))
    print("Goal:");  show_state(normalise(goal))
    plan = blocks_world(start, goal)
    for i, (action, state) in enumerate(plan, 1):
        print(f"Step {i}: {action}")
        show_state(state)


if __name__ == "__main__":
    program10()
```